— 同济大学汽车与能源学院本科生课程 —

数据结构与算法

Week 2

张元建

项目发布

课程作业介绍

它是什么?

KDD Cup 是与 ACM SIGKDD 会议配套举办的年度数据挖掘与知识发现竞赛，由 ACM SIGKDD 组织，长期被视为数据挖掘领域很有影响力的国际赛事之一。

谁来承办?

KDD Cup 2017 由 Alibaba Cloud 团队承办，题目名称为 Highway Tollgates Traffic Flow Prediction。

规模有多大?

官方结果页给出：共有 3547 支队伍参与；比赛分为 Travel Time Prediction 与 Volume Prediction 两个子赛道。

Background

高速公路收费站是交通网络中众所周知的瓶颈。高峰时段，收费站前的长队会给交通管理部门带来巨大压力。因此，需要采取有效的预防措施来应对这一挑战。这些措施包括加快收费流程和优化未来的交通流量。加快收费流程可以简单地通过增派临时收费员来开放更多车道。优化未来的交通流量可以通过对上游路口的交通信号灯进行自适应调整来实现。只有当交通管理部门能够获得可靠的未来交通流量预测时，这些预防措施才能发挥作用。例如，如果预测到下一小时内交通拥堵严重，那么交通管理人员可以立即增派收费员和/或在上游路口分流车辆。

交通流量模式会受到天气状况、节假日、时间等各种随机因素的影响而变化。预测未来交通流量和预计到达时间 (ETA) 一直是一个公认的挑战。来自Waze（美国）或高德地图（中国）等移动应用程序的前所未有的大量交通数据可以帮助我们应对这一挑战。如果参赛者能够设计出可靠的方法来预测未来的交通流量和到达时间，那么交通管理部门就可以利用大数据和算法来减少收费站的拥堵。

课程作业介绍（车流量预测）

Task： 车流量预测

目标：对于每一个 20 分钟的时间窗口，请预测收费站 1、2、3 的入口和出口交通流量。

需要注意的是，收费站 2 只允许车辆驶入高速公路，而其他收费站则允许双向通行，也就是既有驶入，也有驶出。

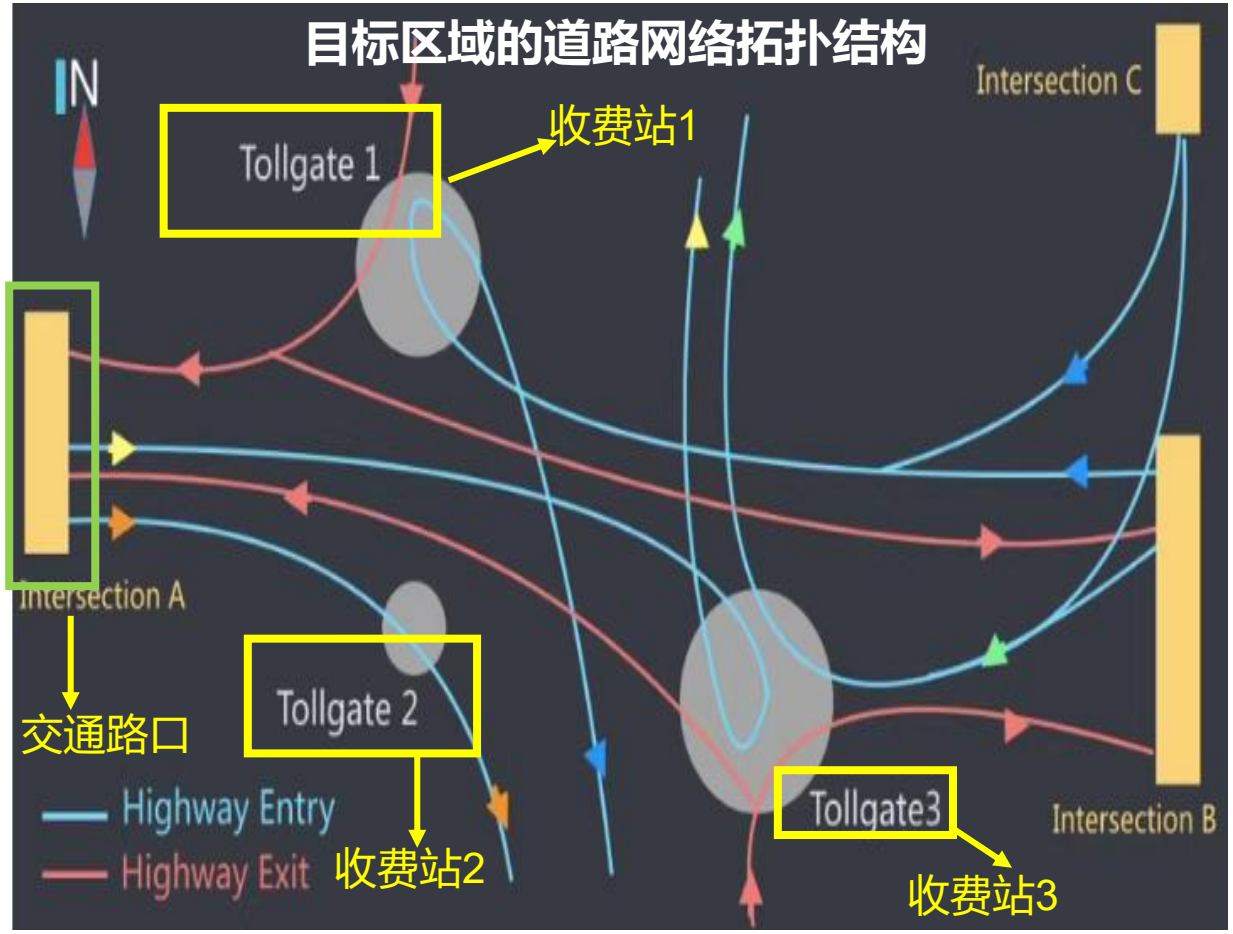
因此，我们总共需要预测 5 个“收费站—方向”组合 的交通流量。

也就是说，这不是一个单点问题，而是一个由多个收费站和多个路口共同组成的交通网络。这说明该任务不是“单点分类”，而是典型的时序回归 / 预测问题。

蓝线：驶入高速 红线: 驶出高速

提交格式

Field	*Type*	*Description*
tollgate_id	string	tollgate ID
time_window	string	e.g., [2016-09-18 08:40:00,2016-09-18 09:00:00)
direction	string	0: entry, 1: exit
volume	int	total volume



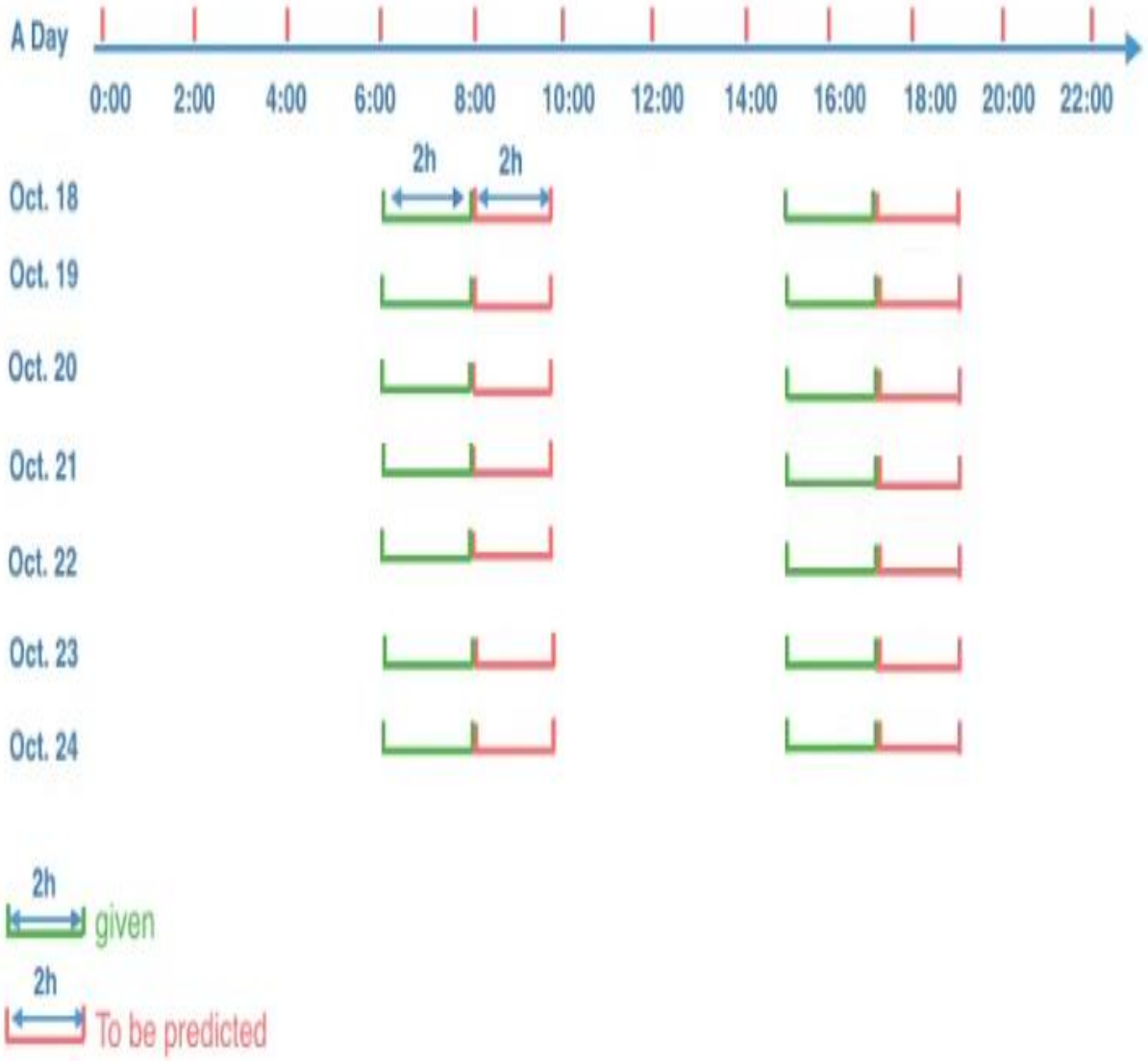
课程作业介绍（车流量预测）

- 需要对 **10 月 18 日至 10 月 24 日** 这段时间内特定高峰时段的交通情况进行预测。也就是说我们需预测图中**红色时间段**内接下来的交通情况，也就是每天 **08:00 到 10:00** 以及 **17:00 到 19:00** 这两个时间段内的交通，并且预测粒度为**每 20 分钟一个时间间隔。**

- **初始训练集**包含 **9 月 19 日至 10 月 17 日** 期间采集的数据。**10 月 18 日至 10 月 24 日** 的额外训练数据将会被加入到预测任务中

- 在**测试数据**中，会得到图 中**绿色时间段**内的交通数据，也就是 **06:00 到 08:00** 和 **15:00 到 17:00** 这两个时间段的数据。我们可以把这些信息作为对**接下来两个小时交通情况的先行指示信息**，用来帮助预测后面两个小时的交通。

- 注意：在进行预测时，**并不被限制只能使用前两个小时的数据，**但是，每一次预测都**只能使用所预测时间窗口之前的交通数据，**例如，参赛者**不允许使用 10 月 20 日的交通数据去预测 10 月 19 日的交通情况。**



课程作业介绍（车流量预测）

Evaluation Metrics:

$$MAPE = \frac{1}{C}\sum_{c=1}^{C}\left(\frac{1}{T}\sum_{t=1}^{T}\left|\frac{f_{ct} - p_{ct}}{f_{ct}}\right|\right)$$

C 表示收费站—方向组合的数量（如前所述：1号收费站入口、1号收费站出口、2号收费站入口、3号收费站入口和3号收费站出口）
T 为测试期间的时间窗口数量
f_{ct} 为特定收费站方向对 c 在时间窗口 t 内的实际交通量
p_{ct} 分为特定收费站方向对 c 在时间窗口 t 内的预测交通量。

评价指标：MAPE（平均绝对百分比误差）—— 误差越小越好，实际值较小时会更敏感，因此异常值与噪声处理非常重要。

数据描述

这里使用的道路网络（图 1）是由彼此连接的道路链路（图 3）构成的一个**有向图**。网络中的一条**路径**（route）（图 4）可以表示为一串按顺序连接的**链路**（links）。

对于每一条道路链路来说，车辆流量会从一个或多个“流入该链路的道路链路”进入，并流向一个或多个“从该链路流出的道路链路”。

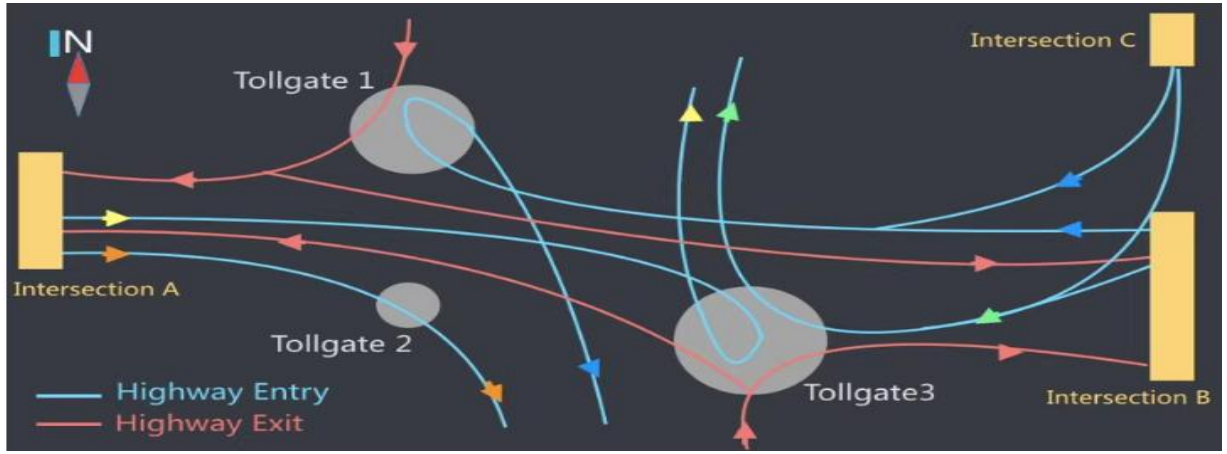


Figure1

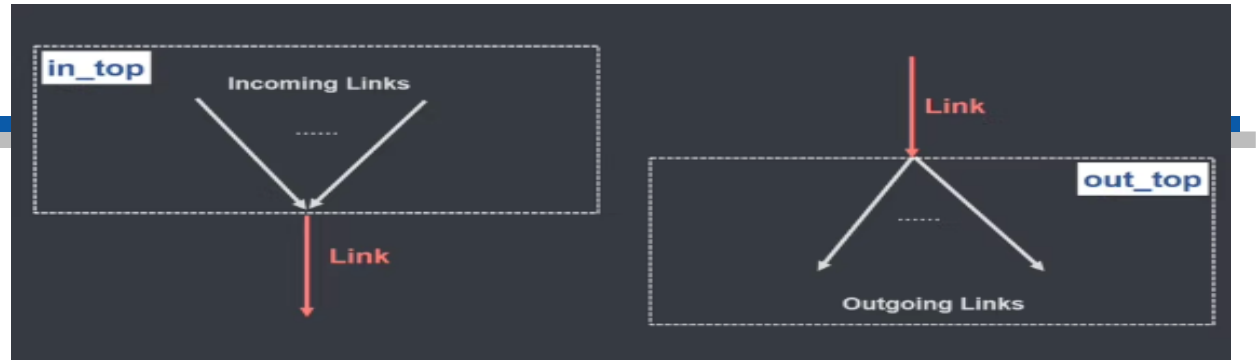


Figure3

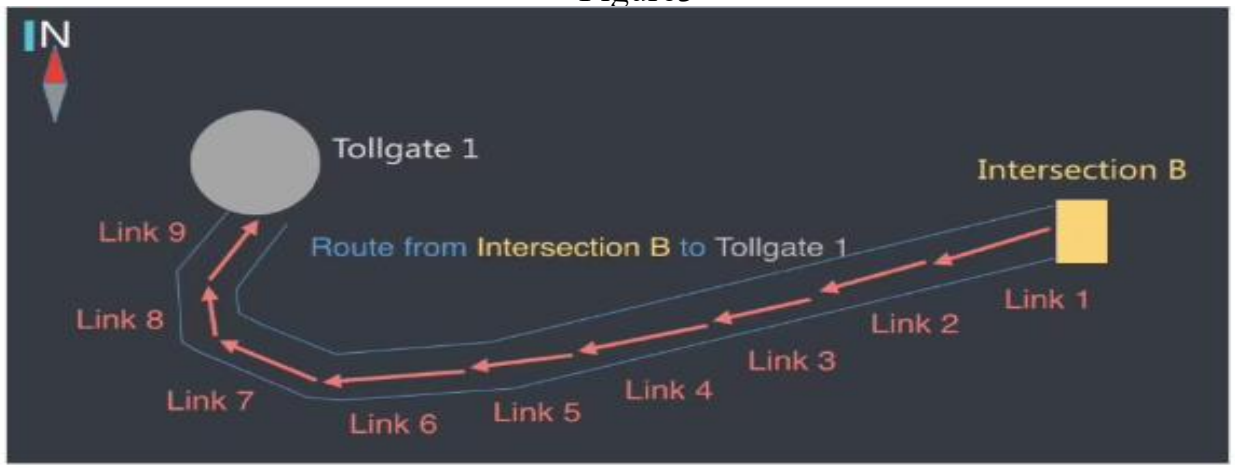


Figure 4. Link Sequence for the Route from Intersection B to Tollgate 1

Table 3. Road Link Properties

Field	*Type*	*Description*
link_id	string	link id
length	float	length (meter)
width	float	length (meter)
lanes	int	number of lanes
in_top	string	incoming road link(s), separated by comma (as shown in Figure 3)
out_top	string	outgoing road link(s), separated by comma (as shown in Figure 3)
lane_width	float	lane width (meter)

数据描述

从道路交叉口到高速收费站的车辆行驶路线选择是有限的。

对于每一个“路口—收费站”组合，我们只选取其中最重要的一条路线记录到表 4 中。

例如，图 4 展示了从 Intersection B 到 Tollgate 1 的一条路线，这条路线由 9 条连续的道路链路组成。

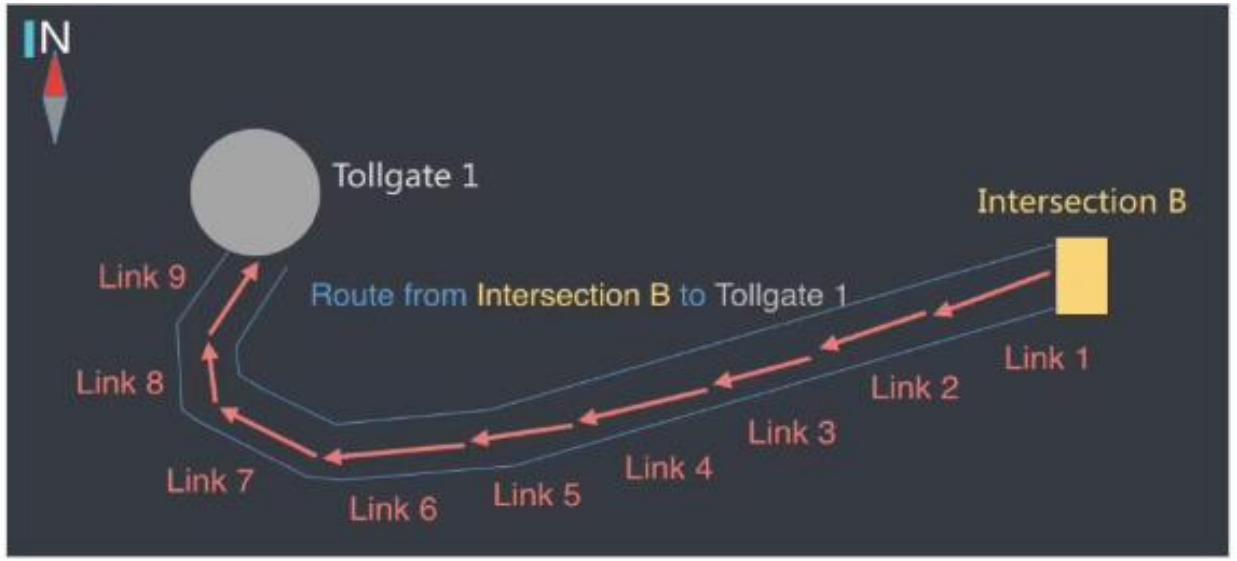


Figure 4. Link Sequence for the Route from Intersection B to Tollgate 1

Table 4. Vehicle Routes from Intersections to Tollgates

Field	Type	Description
intersection_id	string	intersection ID
tollgate_id	string	tollgate ID
link_seq	string	a sequence of link IDs from the intersection to the tollgate separated by commas (shown in Figure 4)

字符串类型，从路口到收费站的一串 link 编号，按顺序排列，多个编号之间用逗号分隔

数据描述

除了静态的路网结构与路线信息之外，项目还提供三类随时间变化的动态数据：车辆轨迹数据、收费站通行数据和天气数据。

Table 5. Vehicle Trajectories Along Routes

沿路线的车辆轨迹数据

Field	Type	Description
intersection_id	string	intersection ID
tollgate_id	string	tollgate ID
vehicle_id	string	vehicle ID
starting_time	datetime	time point when the vehicle enters the route
travel_seq	string	trajectory in the form of a sequence of link traces separated by ";", each trace consists of link id, enter time, and travel time in seconds, separated by "#"
travel_time	float	the total time (in seconds) that the vehicle takes to travel from the intersection to the tollgate

车辆进入这条路线的时间 ← *starting_time*

轨迹序列，以若干个 link 轨迹组成 ← *travel_seq*

从路口到收费站总共花费的时间 ← *travel_time*

数据描述

除了静态的路网结构与路线信息之外，项目还提供三类随时间变化的动态数据：车辆轨迹数据、收费站通行数据和天气数据。

Table 6. Traffic Volume through the Tollgates

通过收费站的交通流量数据

Field	Type	Description
time	datetime	the time when a vehicle passes the tollgate
tollgate_id	string	ID of the tollgate
direction	string	0: entry, 1: exit
vehicle_model	int	this number ranges from 0 to 7, which indicates the capacity of the vehicle (bigger the higher)
has_etc	string	does the vehicle use ETC (Electronic Toll Collection) device? 0: No, 1: Yes
vehicle_type	string	vehicle type: 0-passenger vehicle, 1-cargo vehicle

整数类型，取值范围为 0 到 7，表示车辆容量等级，数值越大表示车辆容量越大

是否有etc

车辆类型：客车还是货车

数据描述

除了静态的路网结构与路线信息之外，项目还提供三类随时间变化的动态数据：车辆轨迹数据、收费站通行数据和天气数据。

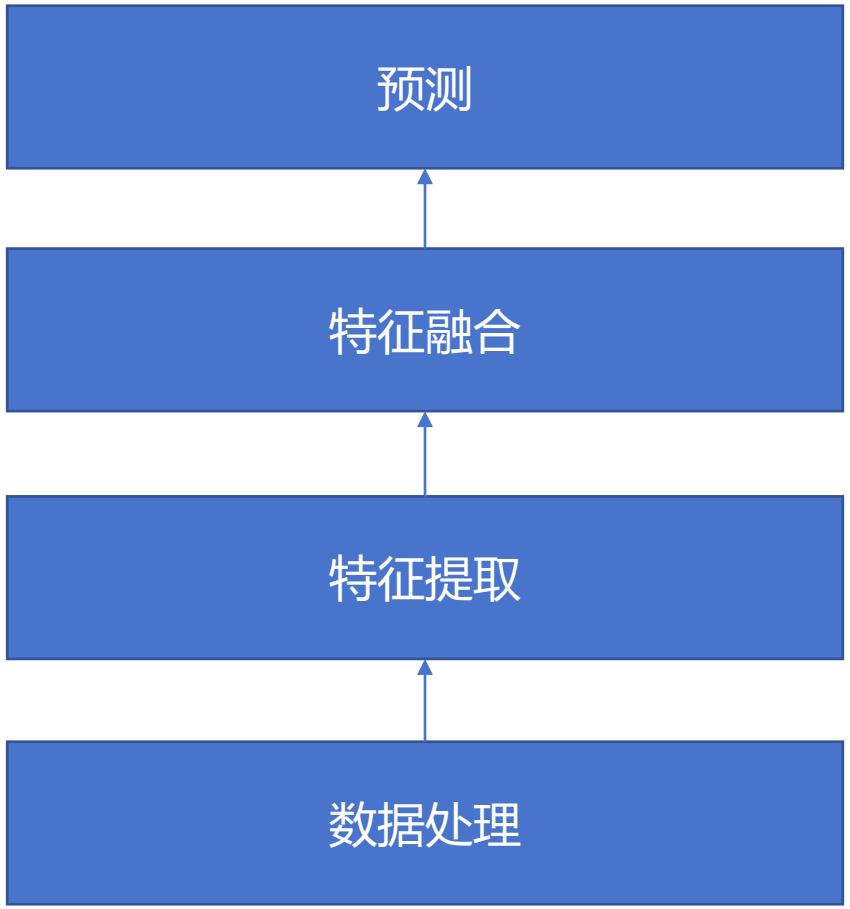
目标区域每 3 小时一次的天气数据

Table 7. Weather Data (every 3 hours) in the Target Area

Field	*Type*	*Description*
date	date	date
hour	int	hour
pressure	float	air pressure (hPa: Hundred Pa)
sea_pressure	float	sea level pressure (hPa: Hundred Pa)
wind_direction	float	wind direction (°)
wind_speed	float	wind speed (m/s)
temperature	float	temperature (℃)
rel_humidity	float	relative humidity
precipitation	float	precipitation (mm)

表 3 和表 4 是静态数据，只在训练集中提供；表 5、表 6、表 7 是动态数据，在训练集和测试集中都会提供。

课程作业介绍—流程

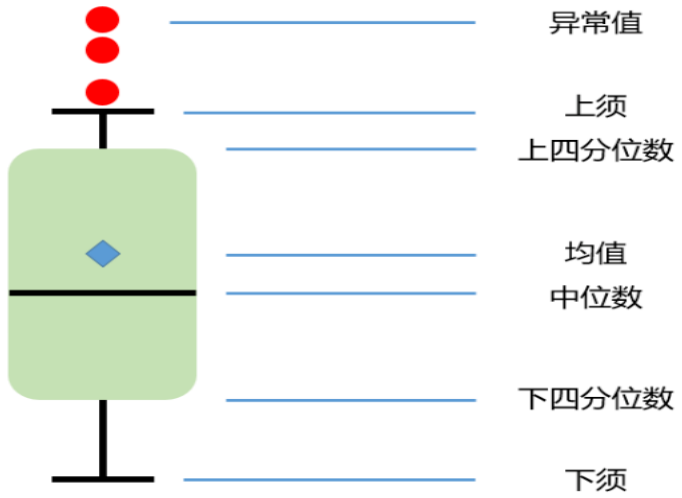


数据处理---异常值清洗与平滑

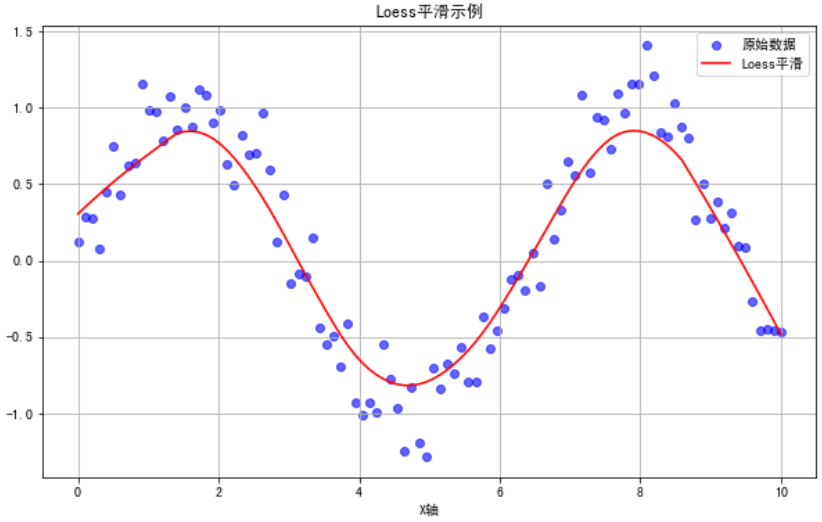
作用： 去除不合理记录，降低噪声对模型的干扰

常见异常：
- 极大或极小流量值
- 采集错误导致的突变
- 与前后时间窗口差异过大的点。

常见方法：
- 阈值剔除
- 3σ原则
- 箱线图法
- 邻域平滑



箱线图找异常值



邻域平滑

数据处理---缺失值处理

缺失的可能原因：

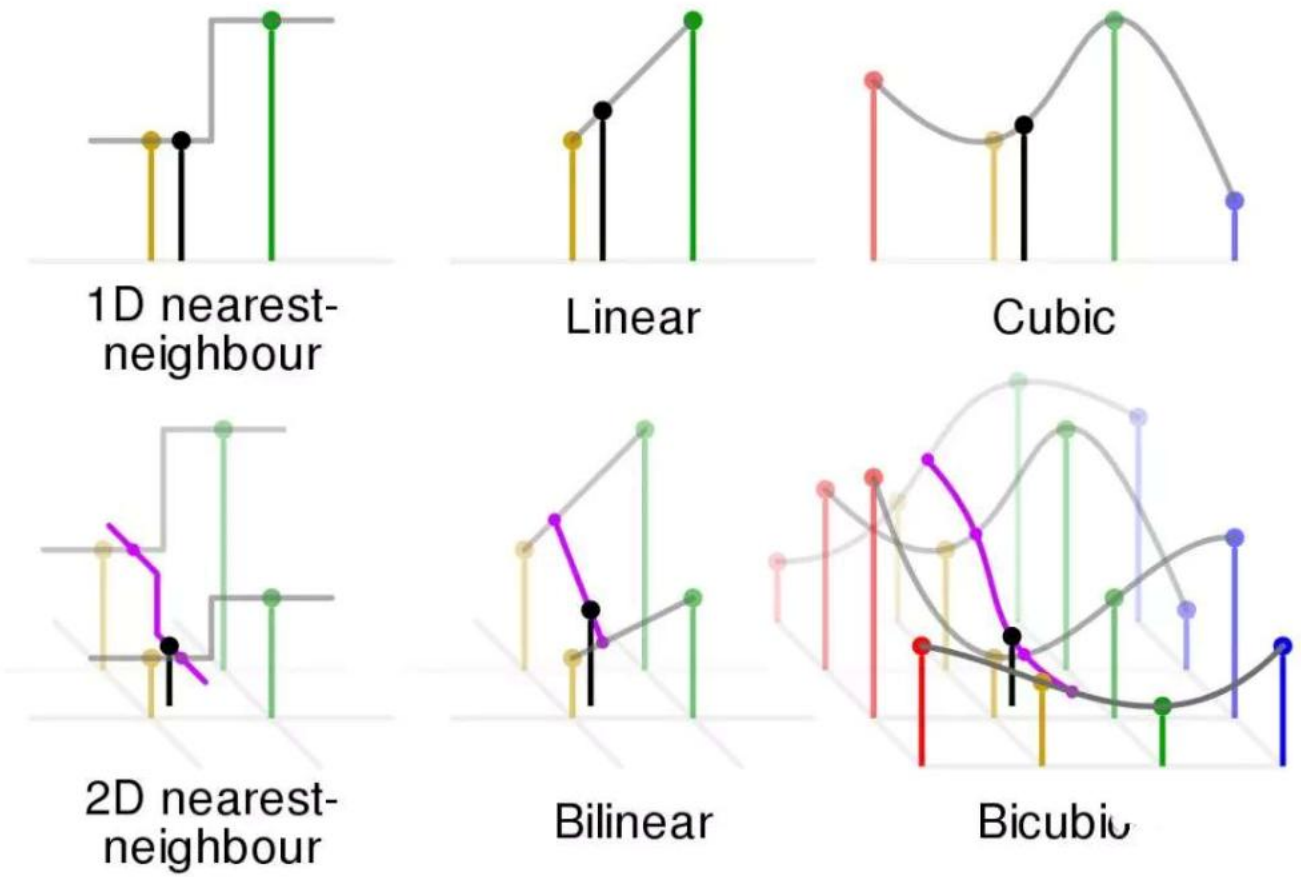
- 某时间片无记录
- 采集设备故障
- 数据同步问题
- 天气数据时间粒度不同

常见处理方法：

- 前向填充
- 后向填充
- 均值/中位数填充
- 线性插值
- 删除严重缺失样本

注意：

缺失值处理应保持时间顺序，不引入未来信息



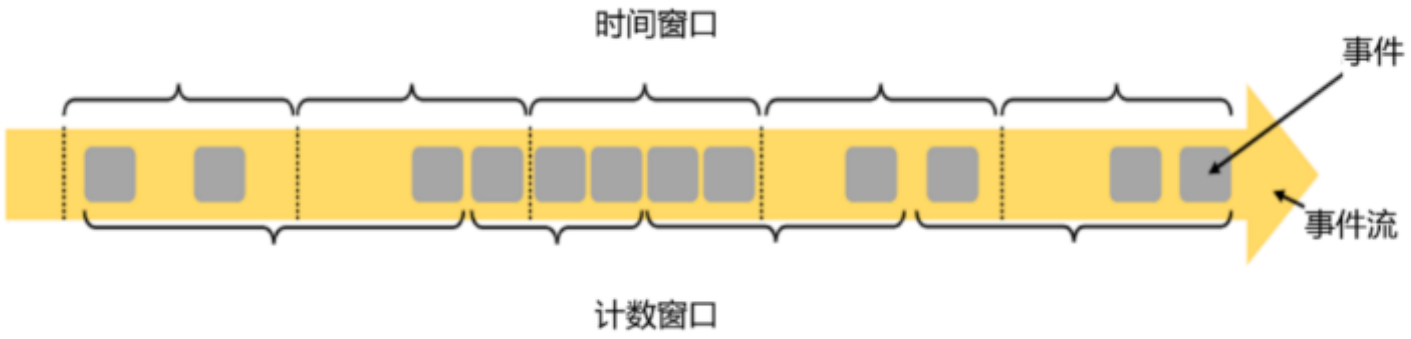
不同插值补全方法

数据处理---时间窗口切片

核心思想：
将连续的车辆记录按固定时间粒度切分为统一样本单元

本项目设置：
•以 20 分钟为一个时间窗口
•每个窗口统计车流量、车辆类型数量等信息
•形成窗口级特征表

示例：
08:00–08:20
08:20–08:40
08:40–09:00



时间窗口

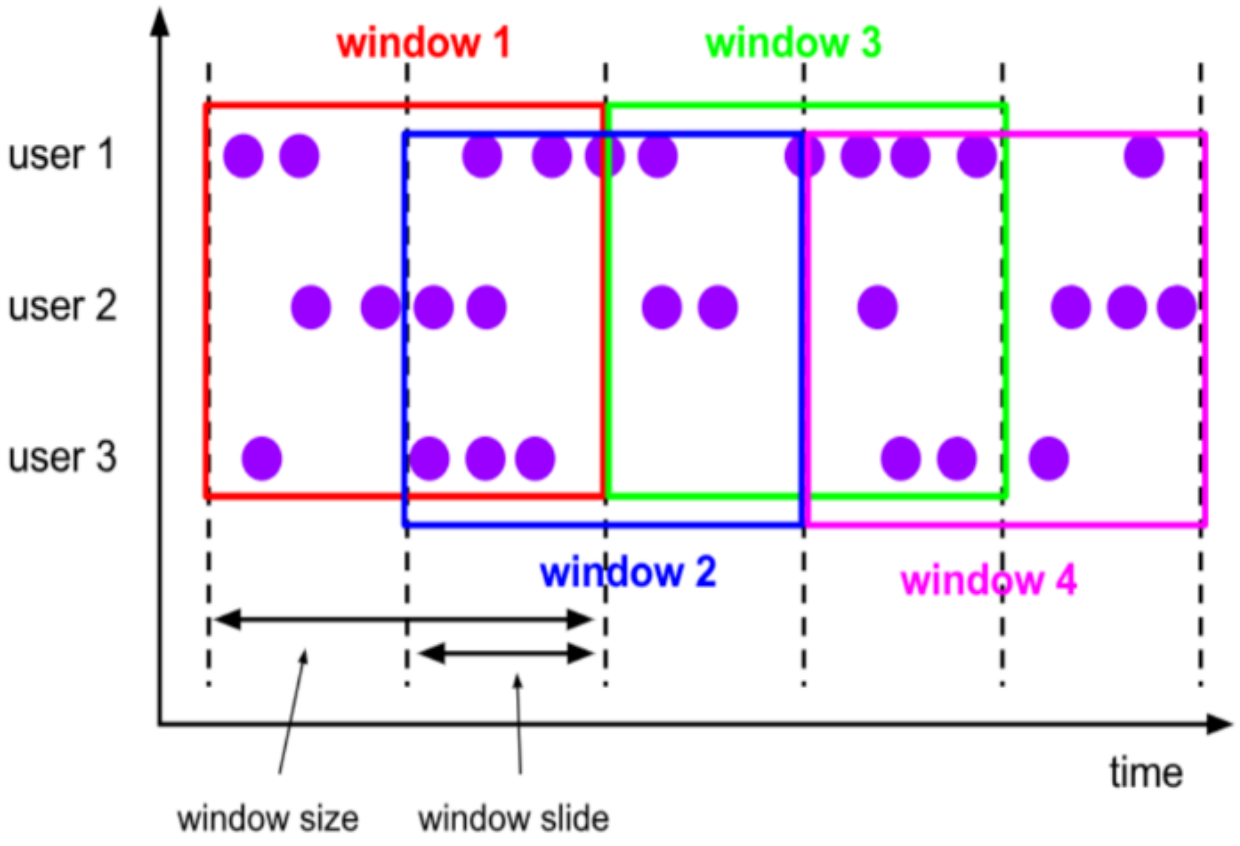
数据处理---滑动窗口构造样本

目的：
将时间序列问题转化为监督学习问题

构造方式：
•输入：前6个连续窗口
•输出：后1个或多个未来窗口

示例：
[a,b,c,d,e,f] → g
[b,c,d,e,f,g] → h
[c,d,e,f,g,h] → i

意义：
用历史模式预测未来流量



滑动时间窗口

特征提取---Lasso与Ridge

LASSO：
- L1 正则化
- 可做特征选择
- 产生稀疏解

$$\sum_{i=1}^{n}(Y_i - \sum_{j=1}^{p} X_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

LASSO

Ridge：
- L2 正则化
- 抑制过拟合
- 处理共线性更稳定

$$\sum_{i=1}^{n}(y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Ridge

在本项目中的作用：
作为基线模型与特征筛选工具

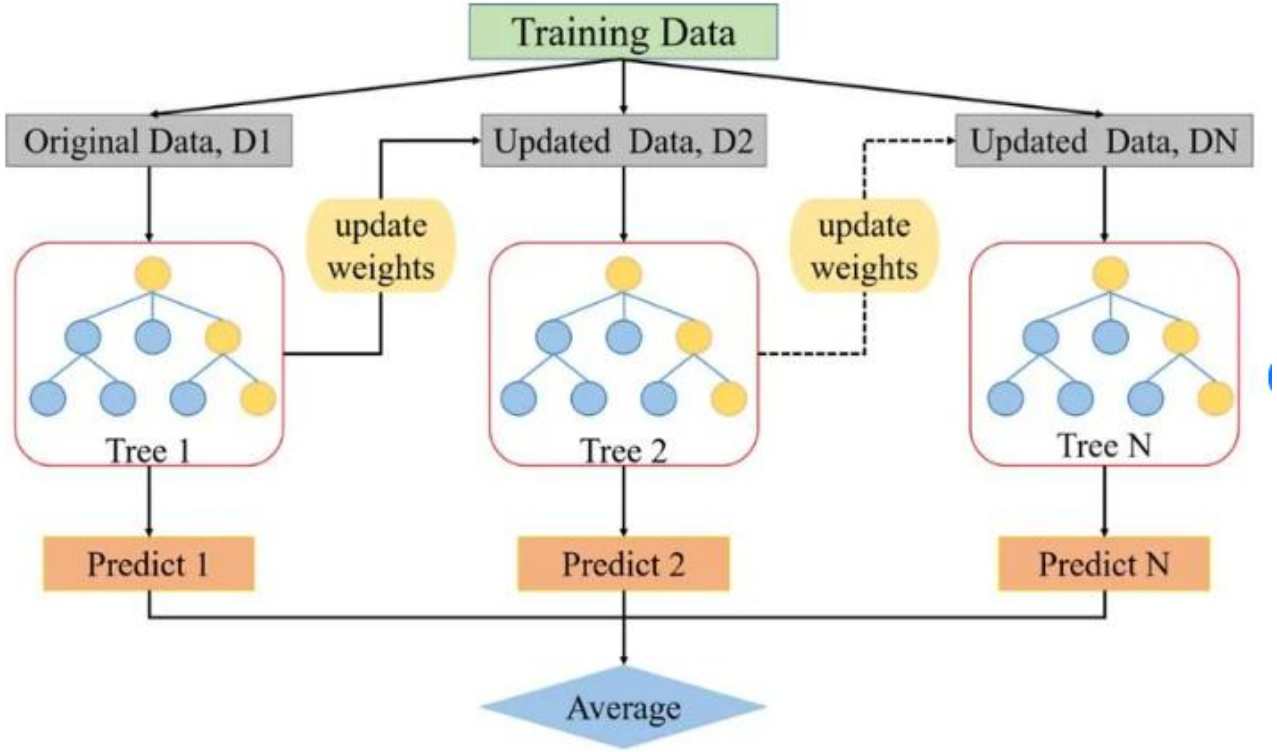
特征提取---GBDT

GBDT 特点：

- 基于提升树
- 逐步减少残差
- 能学习非线性关系
- 适合结构化表格数据

在本项目中的作用：

学习历史特征、时间特征、天气特征与未来流量之间的映射关系



特征提取---Random Forest 与 ExtraTrees

Random Forest：

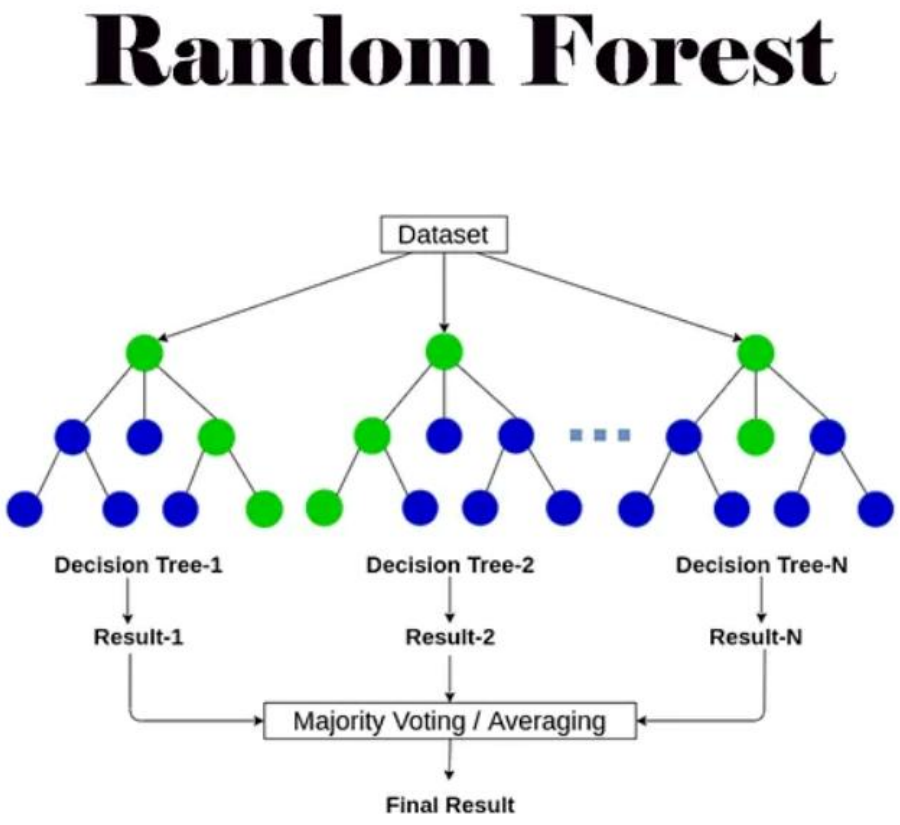
- 多棵决策树并行训练
- 随机采样与随机特征选择
- 结果稳定，抗过拟合

ExtraTrees：

- 比随机森林更随机
- 分裂点随机性更强
- 训练速度更快

作用：

用于特征重要性评估和稳健预测



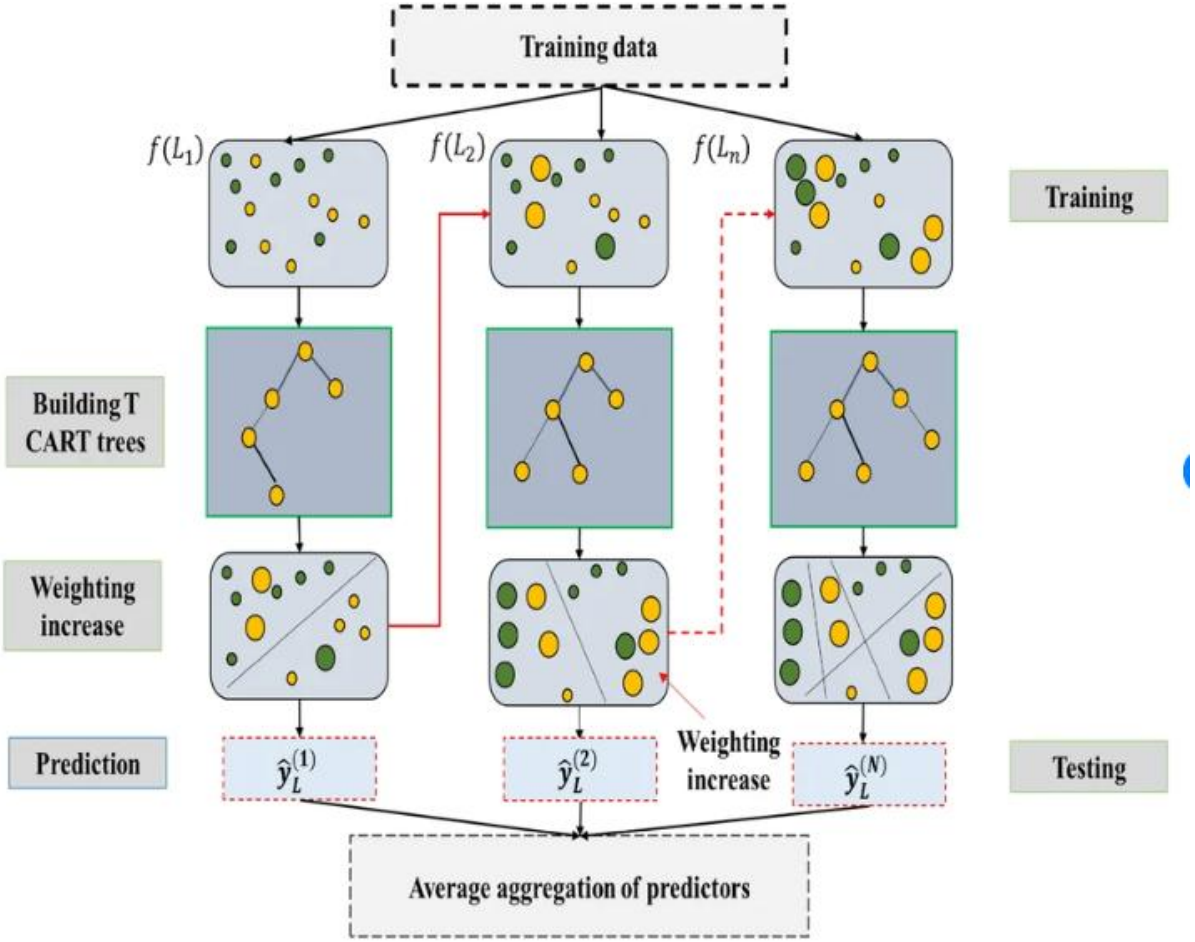
特征提取--- XGBoost

XGBoost 特点：

- GBDT 的强化版本
- 加入正则化
- 训练效率高
- 支持缺失值处理
- 表格数据表现强

在本项目中的作用：

作为核心回归模型之一，用于预测未来 traffic volume



特征融合--- 特征拼接

方法：
将多种特征直接连接为统一向量

可拼接的特征：
- 历史流量特征
- 时间特征
- 天气特征
- 拓扑特征

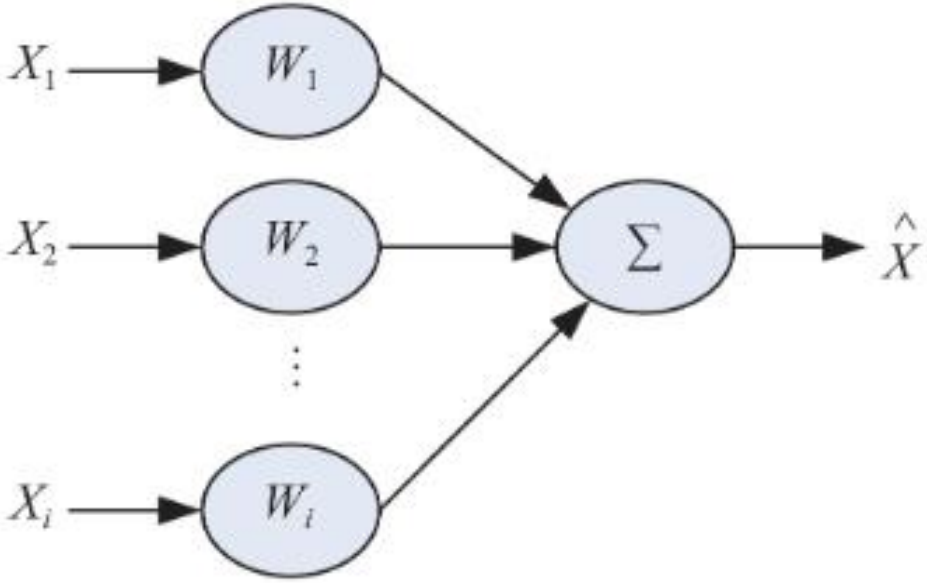
优点：
简单直接，易于实现

特征融合--- 加权融合

基本思想：
不同特征的重要性不同，可给予不同权重

示例：
- 短期流量特征权重大
- 天气特征权重中等
- 弱相关特征权重较小

作用：
突出关键特征，减少噪声干扰

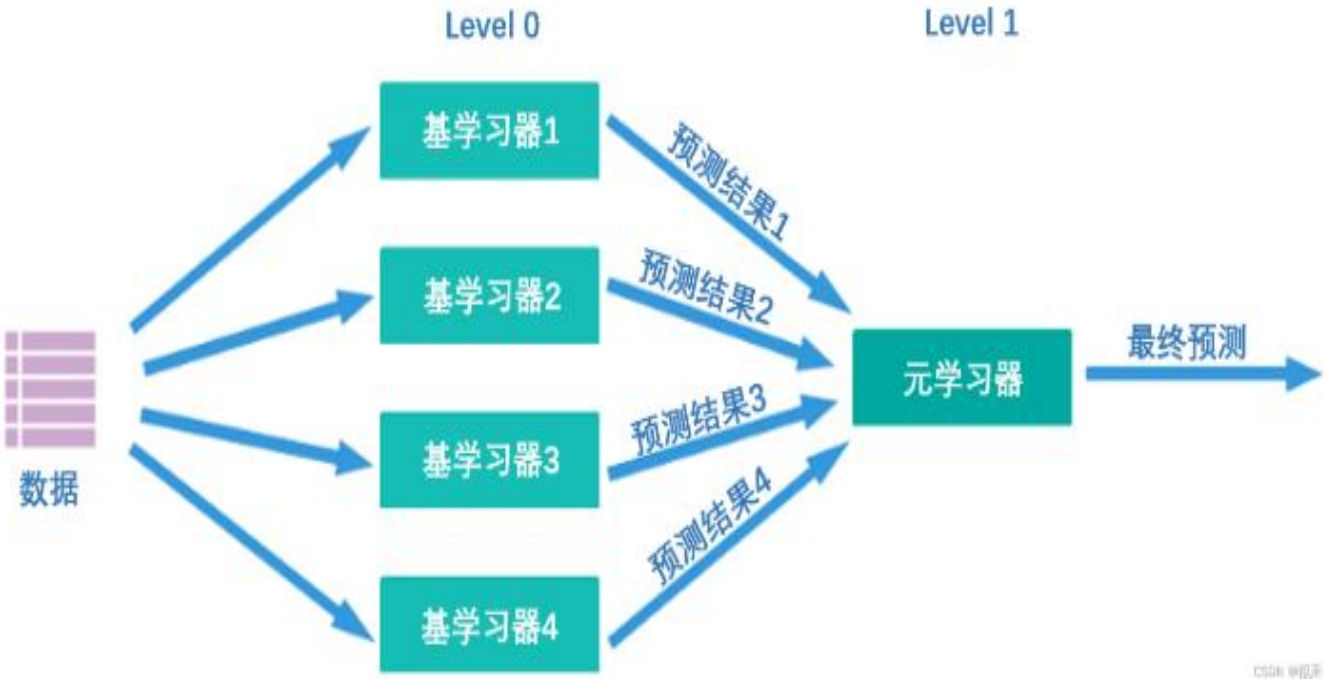


特征融合--- Stacking融合

基本思想：
先训练多个基模型，再用二层模型融合它们的输出

典型流程：
- 第一层：GBDT / XGBoost / Random Forest
- 第二层：XGBoost / Linear Model

作用：
融合不同模型的优势，提升整体效果



课程作业介绍

任务描述：https://tianchi.aliyun.com/competition/entrance/231597/information?from=oldUrl
数据集：https://tianchi.aliyun.com/dataset/60
参考项目： https://github.com/ttff96989/KDD-CUP-2017

训练集

测试集

额外训练测试集训练集

课程作业介绍—参考项目介绍

参考项目：https://github.com/celinezhangzhang/KDD-2017

1. 这个项目在干什么

这是一个 **交通流量预测项目**，针对高速收费站场景：

•训练数据：车辆通过收费站的记录

•预测目标：未来 **8:00–10:00、17:00–19:00** 的车流量

•预测粒度：**每20分钟一个值**

•预测对象：3个收费站、5个方向组合

本质上是一个 时序回归问题，但不是长序列深度学习路线，而是典型的 特征工程 + 集成学习 方案。

2. 项目整体思路

（1）先把原始过车记录转成时间片流量

把车辆通行日志按 **20分钟切片**，统计每个时间片里的：总车流量；不同载重区间车辆数等，然后构造成监督学习样本。

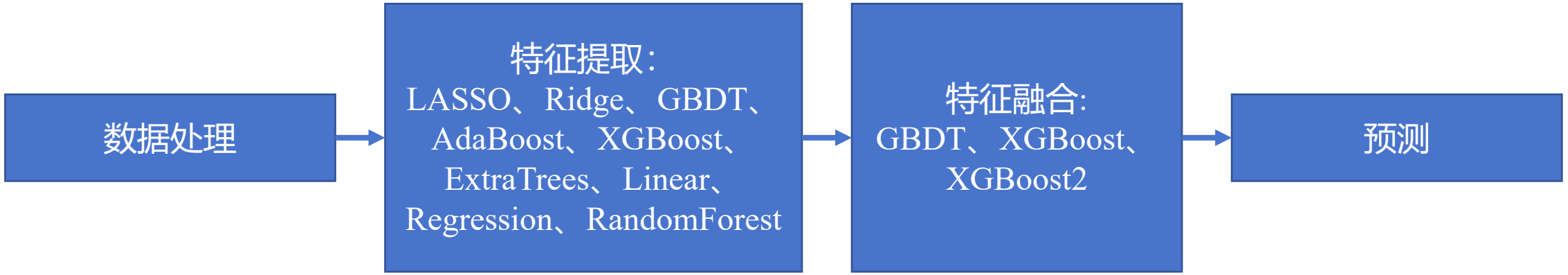
（2）用前 2 小时预测后 2 小时

以连续 **6个20分钟片段** 作为输入特征，预测之后若干个20分钟片段的车流量。

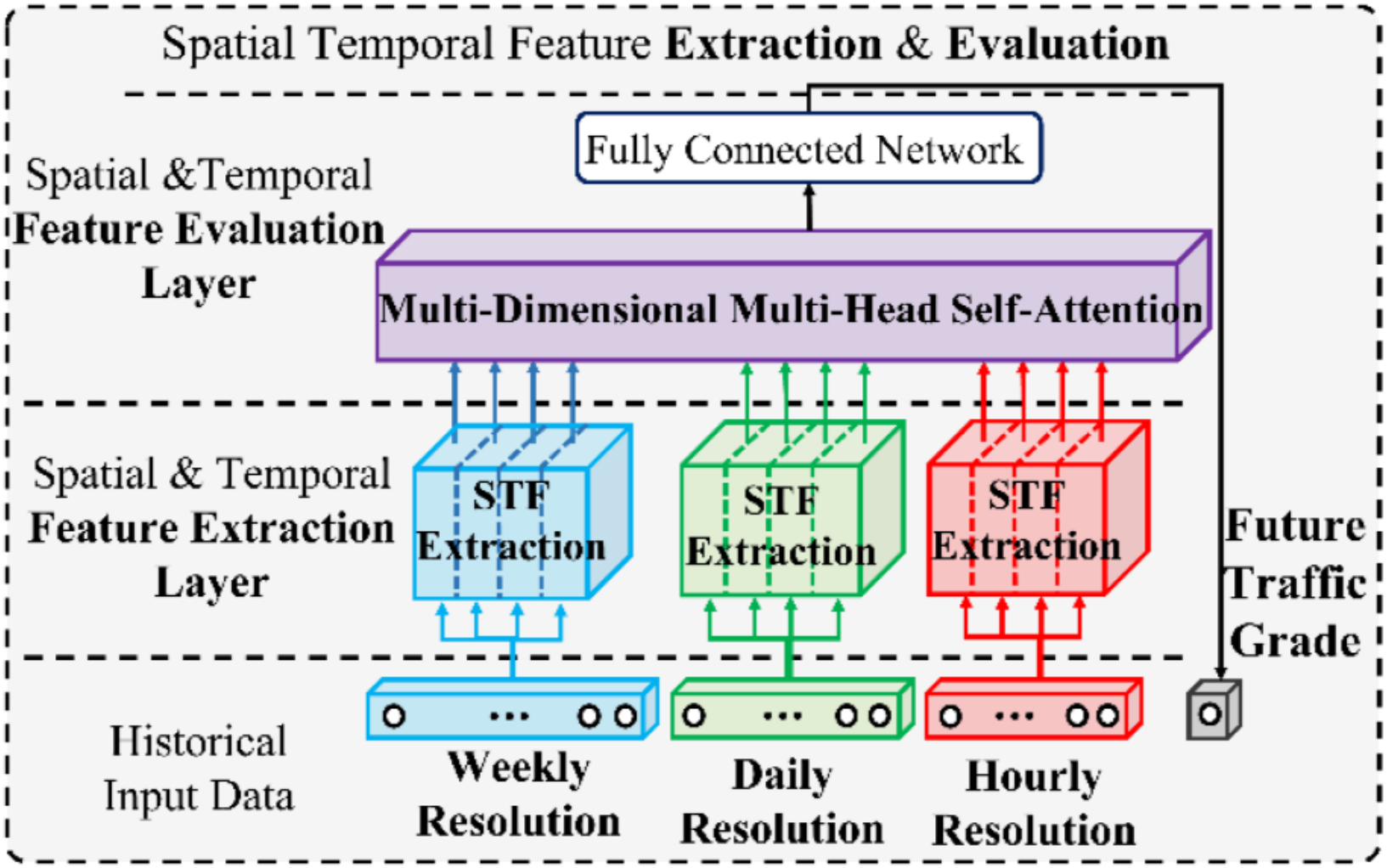
就是一个滑动窗口建模：

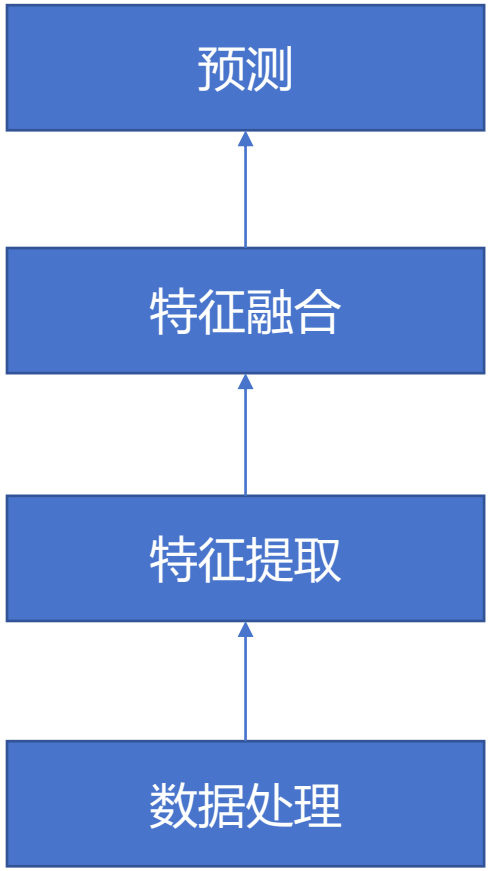
[a,b,c,d,e,f]→g；[b,c,d,e,f,g]→h；[c,d,e,f,g,h]→i

3. 项目使用的方法

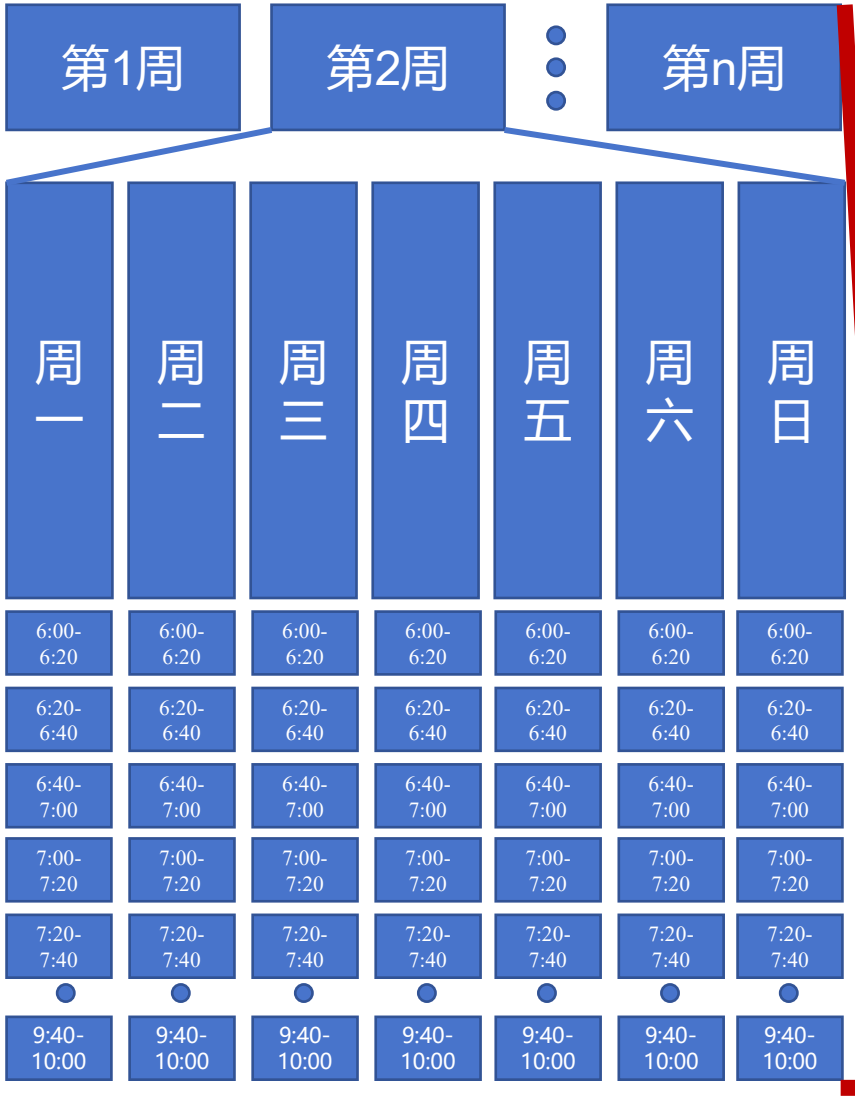


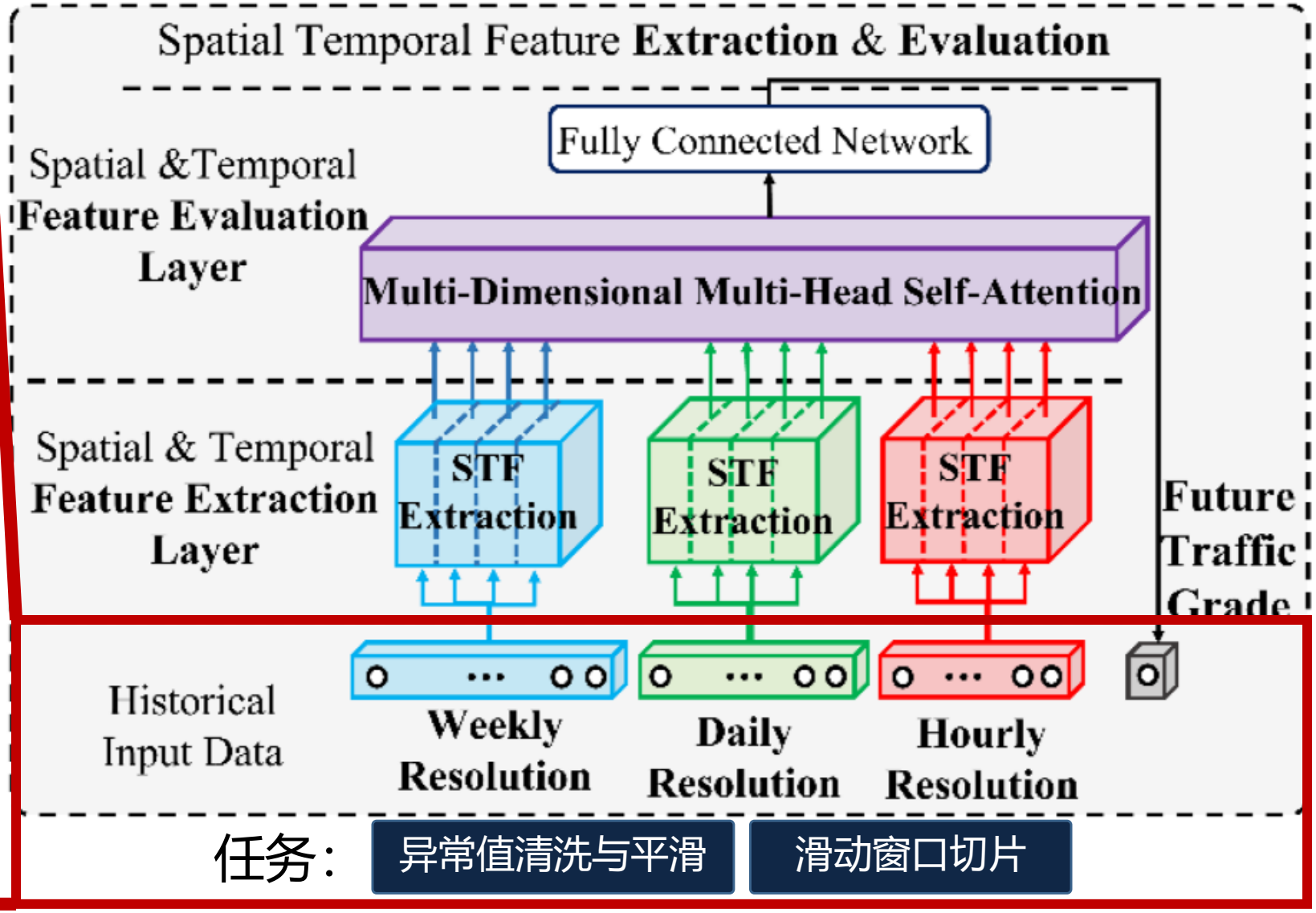
课程作业介绍—流程


Spatial Temporal Feature Extraction & Evaluation
Fully Connected Network
Spatial &Temporal Feature Evaluation Layer
Multi-Dimensional Multi-Head Self-Attention
Spatial & Temporal Feature Extraction Layer
STF Extraction
STF Extraction
STF Extraction
Future Traffic Grade
Historical Input Data
Weekly Resolution
Daily Resolution
Hourly Resolution


预测
特征融合
特征提取
数据处理

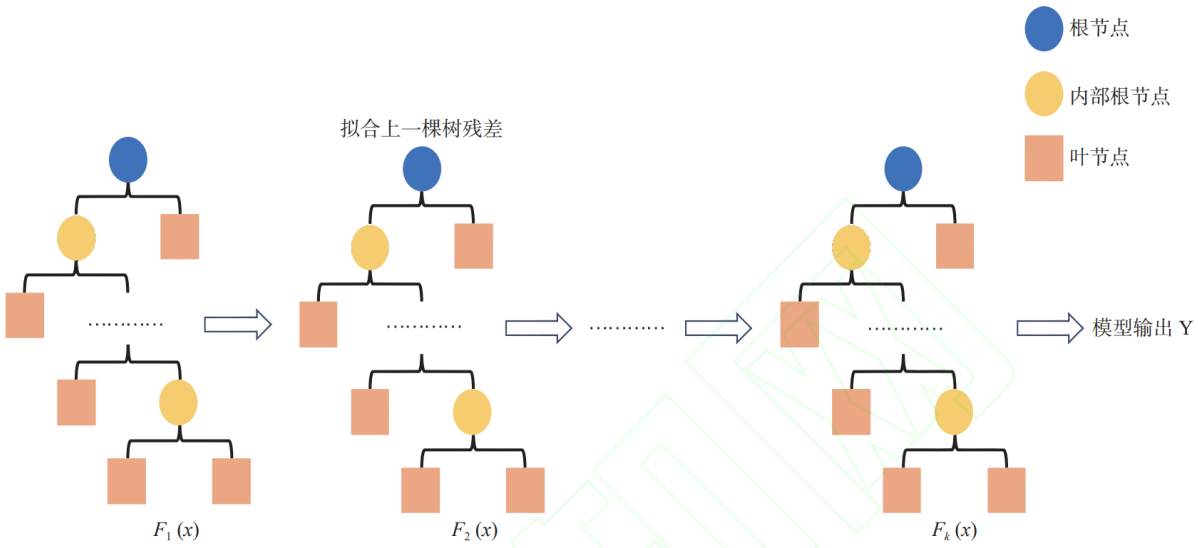
课程作业介绍—数据处理


第1周
第2周
第n周
周一
周二
周三
周四
周五
周六
周日
6:00-6:20
6:20-6:40
6:40-7:00
7:00-7:20
7:20-7:40
9:40-10:00

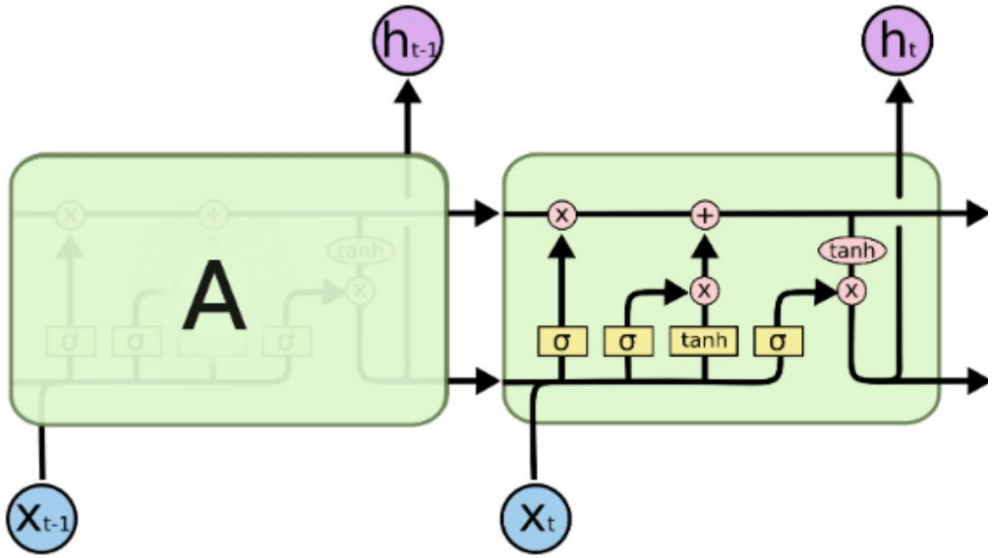

Spatial Temporal Feature Extraction & Evaluation
Spatial &Temporal Feature Evaluation Layer
Fully Connected Network
Multi-Dimensional Multi-Head Self-Attention
Spatial & Temporal Feature Extraction Layer
STF Extraction
STF Extraction
STF Extraction
Future Traffic Grade
Historical Input Data
Weekly Resolution
Daily Resolution
Hourly Resolution
任务：
异常值清洗与平滑
滑动窗口切片

课程作业介绍—特征提取与融合

特征提取与融合任务：让模型自己去学习如何从周、日、小时的数据中提取出最有用的信息。



XGBoost



LSTM

任务：多尺度特征对齐　特征融合与降维

课程作业介绍—汇报

录制视频，7-8分钟，介绍内容如下

团队介绍

- 团队组成及分工等

背景

- 做什么、有什么意义等

数据处理方法

- 数据怎么处理，如何提取更有效的特征

特征提取与融合

- 简单介绍方法，不要陷入模型细节

最终效果

- 该方法实现了什么效果，评价指标

代码讲解

- 代码在哪里实现了什么功能，简要介绍

课程作业介绍—评分

小组评分

- 5 位组员互相打分，默认不允许给自己打分。
- 维度建议：任务完成度、参与积极性、贡献质量、团队协作、责任心与纪律。
- 最终个人成绩 = **小组成绩** × 70% + **个人互评分** × 30%

1. 任务完成度（30分）

•27–30分：按时高质量完成，几乎不用别人返工
•21–26分：基本完成，有少量修改
•15–20分：完成一般，需要较多补充
•0–14分：未按时完成，或完成度很低

2. 参与积极性（20分）

•18–20分：主动提出想法，积极参与全过程
•14–17分：能正常参与讨论和分工
•10–13分：参与较少，多数时候被动
•0–9分：基本不参与

3. 贡献质量（20分）

•18–20分：内容质量高，对作业帮助明显
•14–17分：内容合格，有实际贡献
•10–13分：贡献一般，内容较浅
•0–9分：贡献较少或质量较差

4. 团队协作（15分）

•13–15分：沟通顺畅，合作意识强
•10–12分：能正常配合
•6–9分：协作一般，偶尔影响进度
•0–5分：配合差，影响小组工作

5. 责任心与纪律（15分）

•13–15分：守时认真，反馈及时
•10–12分：总体负责，偶有延迟
•6–9分：提醒后才完成
•0–5分：经常拖延或失联

严禁抄袭！！

严禁抄袭！！

严禁抄袭！！

违者必究！！

违者必究！！

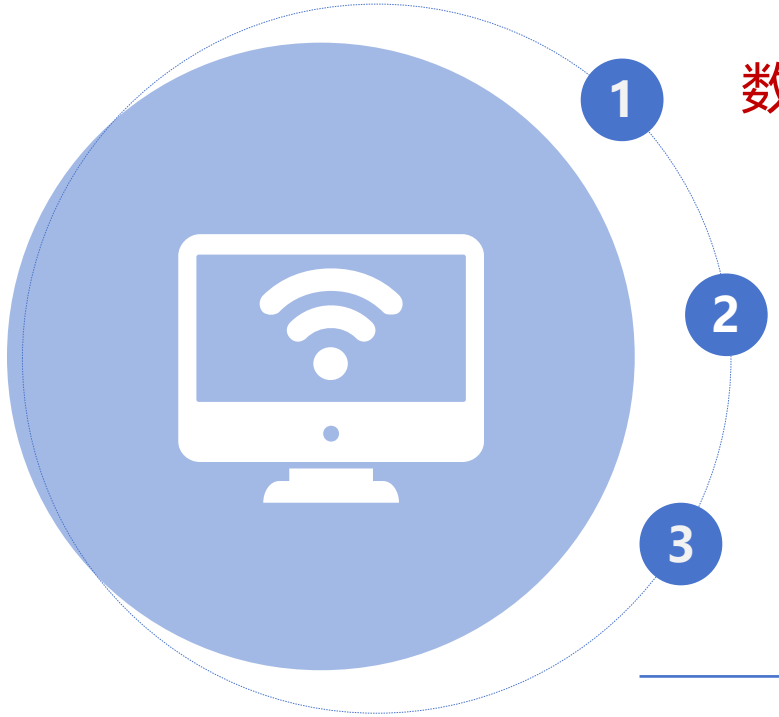
违者必究！！

Recap

- 数据的逻辑结构
  - 线性结构
    - 线性表
    - 栈
    - 队
  - 非线性结构
    - 树形结构
    - 图形结构
- 数据的存储结构
  - 顺序存储
  - 链式存储
- 数据的运算:查找,排序,插入,删除,修改等

Recap：什么是数据结构

数据结构就是一门研究怎样合理组织数据，建立合适的数据存储结构，提高计算机程序执行效率的学科。数据结构主要研究以下三个方面的内容：

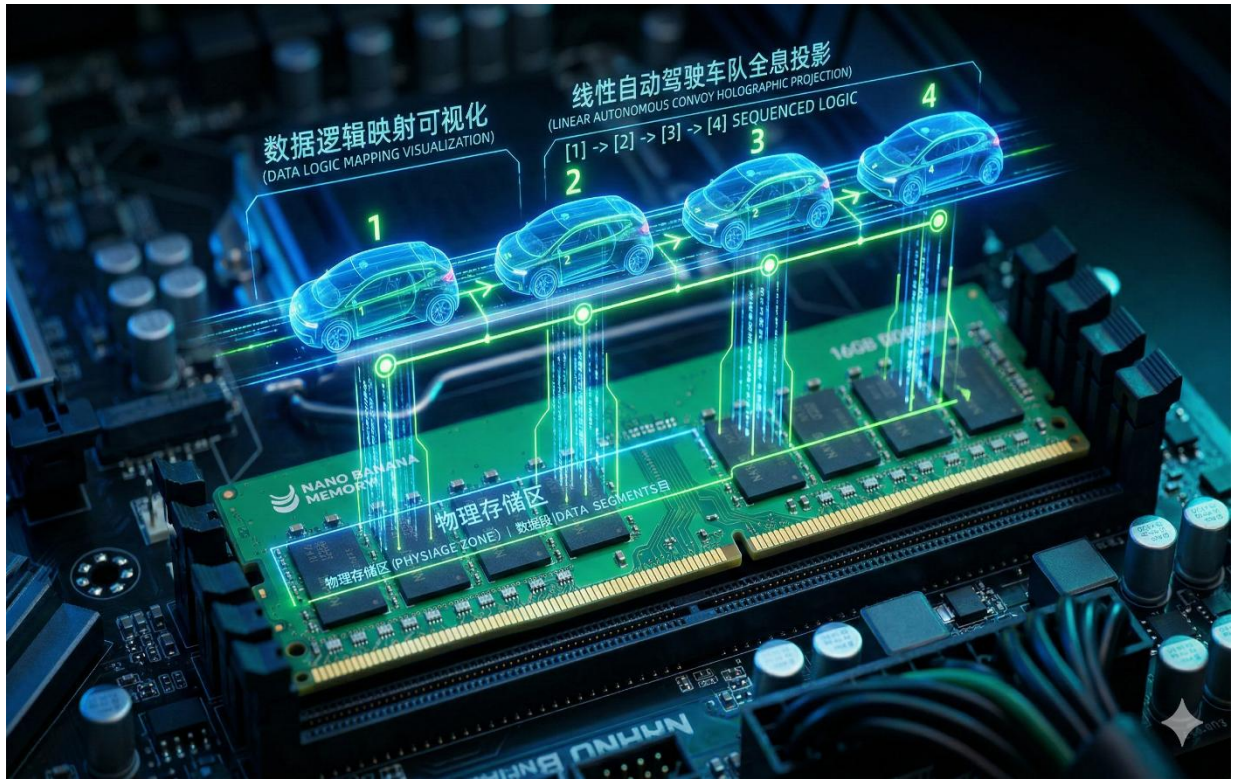


1. 数据元素之间的逻辑关系，也称数据的逻辑结构（Logical Structure）。
2. 数据元素及其关系在计算机存储器内的表示，称为数据的存储结构（Storage Structure）或物理结构。
3. 数据的运算，即对数据施加的操作。

数据的运算定义在数据的逻辑结构上，每种逻辑结构都有一个运算的集合。最常用的检索、插入、删除、更新、排序等运算实际上只是在抽象的数据上所施加的一系列抽象的操作。

Recap：灵魂与肉体：逻辑与物理的完美配合

- **做算法（如路径规划、车队控制）时**： 我们只看逻辑结构。因为这样最简单、最清晰。汽车工程师在画UML图或者写伪代码时，面对的都是逻辑。
- **做底层系统（如车载ECU内存管理、嵌入式C语言）时**： 我们必须考虑物理结构。因为车载芯片的内存极其宝贵！如果内存碎片太多（没有连续车位），我们就只能被迫使用链式结构。
- **总结**： 逻辑结构是“我们想要什么”（车队），物理结构是“计算机能提供什么”（内存条的排队方式）。优秀的数据结构设计，就是在这两者之间找到最优解！！！



细节了解数据结构

数据结构的形式定义

数据结构的形式定义是一个二元组：

B=（K，R）

其中：K是数据元素的有限集，R是K中所有数据元素之间关系的有限集。

例：设数据逻辑结构B=（K，R）

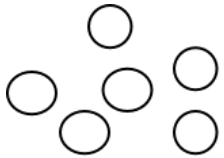
K={k1, k2, ..., ki}

R={R1, R2, ..., Rj}

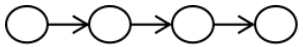
数据的逻辑结构

数据结构(Data Structure)：是指相互之间具有(存在)一定联系(关系)的数据元素的集合。数据元素之间的相互联系(关系)称为逻辑结构。数据元素之间的逻辑结构有四种基本类型：

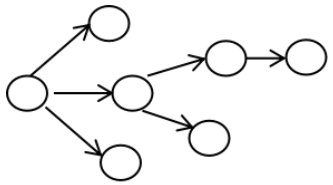
- 集合结构



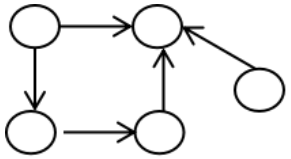
- 线性结构



- 树形结构



- 图形结构



数据的逻辑结构—集合结构

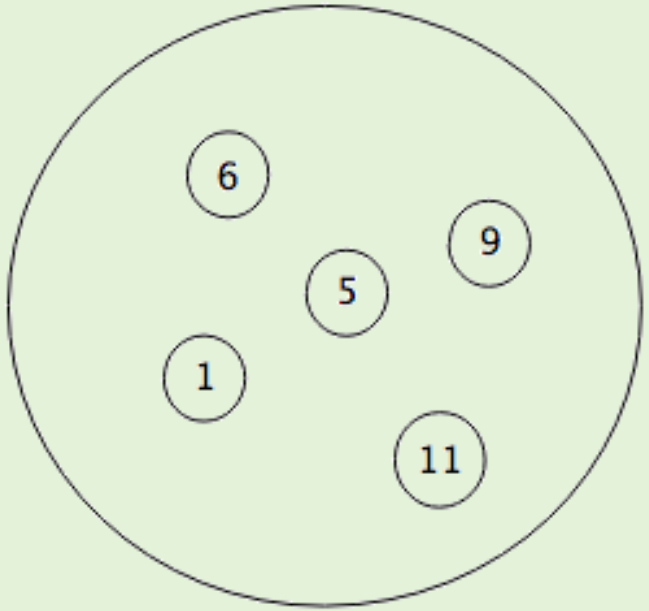
结构中的数据元素之间除了同属于一个集合的关系外，无任何其他关系。

图左所示为一个整数集合，即该集合内的所有元素都是整数。图右为一个实数集合，即该集合内的所有元素都是实数。

B1=（K,R） K={1, 5, 6, 9, 11} R={}

B2=（K,R） K={3.0，6.6，4.8} R={}

数据元素之间的联系：0:0



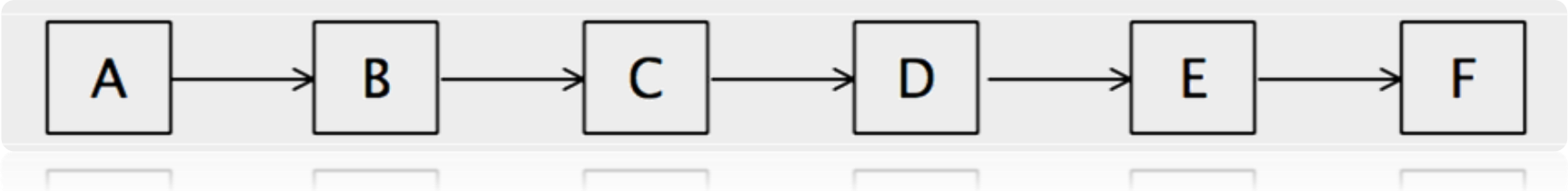
数据的逻辑结构—线性结构

结构中的数据元素之间存在着一对一的线性结构。

线性结构如图所示，数据元素之间有一种先后的次序关系，A是B的前驱， C是B的后继。

B1= （K,R） ； K={A, B, C, D, E, F} ； R={ (A,B), (B,C), (C,D), (D,E), (E,F) }

数据元素之间的联系： 1:1

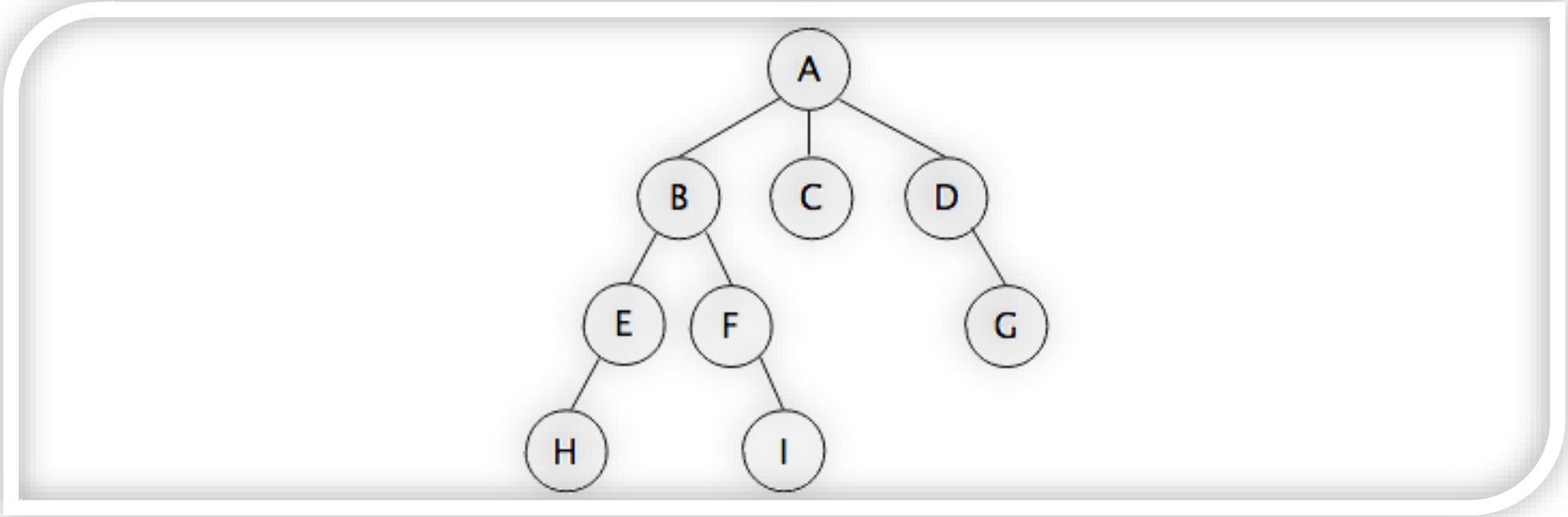


数据的逻辑结构—树形结构

结构中的数据元素之间存在着一对多的层次结构。树形结构如图所示。

B= （K,R）； K={A, B, C, D, E, F, G, H, I }； R={ (A,B), (A,C), (A,D), (B,E), (B,F) , (E,H), (F,I), (D,G) }

数据元素之间的联系： 1:N

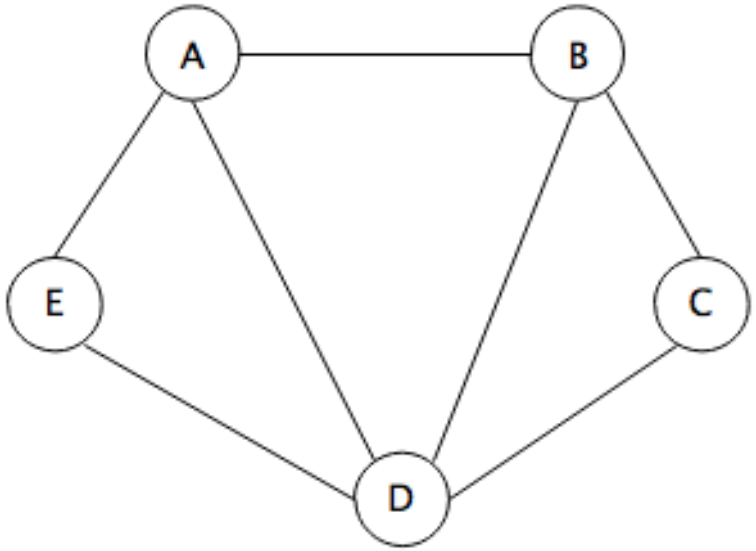


数据的逻辑结构—图结构

结构中的数据元素之间存在着多对多的层次结构。图形结构如图所示。

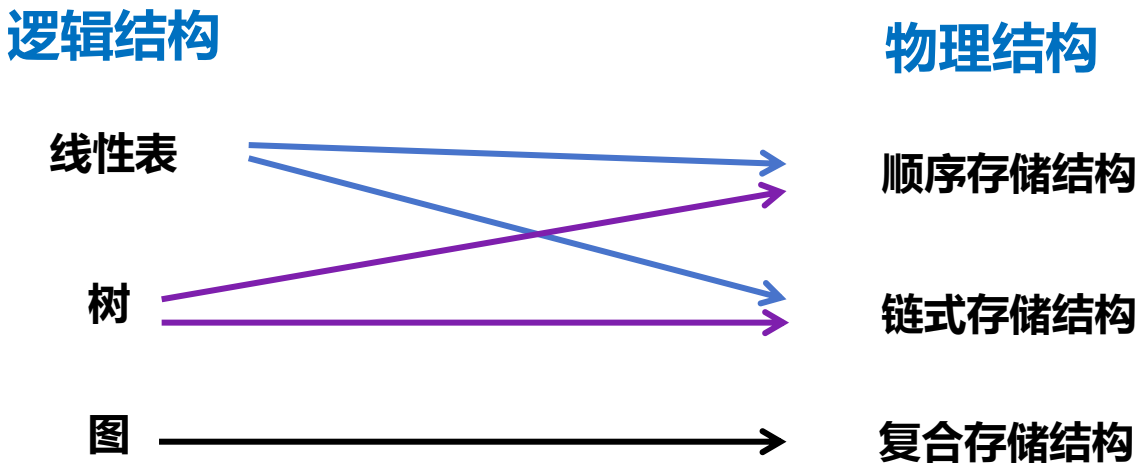
B= （K,R） ； K={ A, B, C, D, E } ； R={ (A,B), (A,E), (A,D), (B,A), (B,D) , (B,C), (C,B), (C,D), (D,A), (D,B), (D,C), (D,E), (E,A) , (E,D) }

数据元素之间的联系： N:N



数据的存储结构

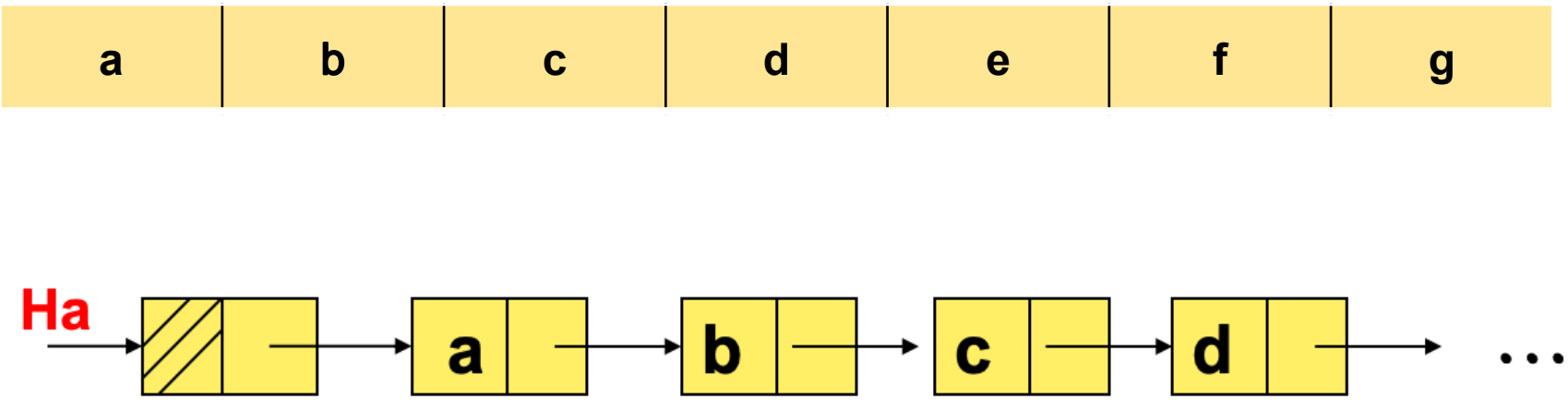
- 数据的存储结构（又叫物理结构）是逻辑结构用计算机语言的实现。
- 数据的存储结构依赖于计算机语言。
  - 顺序存储表示
  - 链接存储表示



逻辑结构与所采用的存储结构

数据的存储结构 — 对比

- 顺序存储——借助元素在存储器中的相对位置来表示数据元素间的逻辑关系
- 链式存储——借助指示元素存储地址的指针表示数据元素间的逻辑关系



数据结构—存储结构【数据的“停车场”】

- **顺序存储（如数组）：** 就像大学里的**标准一字型停车场。**1号、2号、3号车位紧挨着。我们的车队数据必须停在连续的车位里。找完1号车，旁边立刻就是2号车。

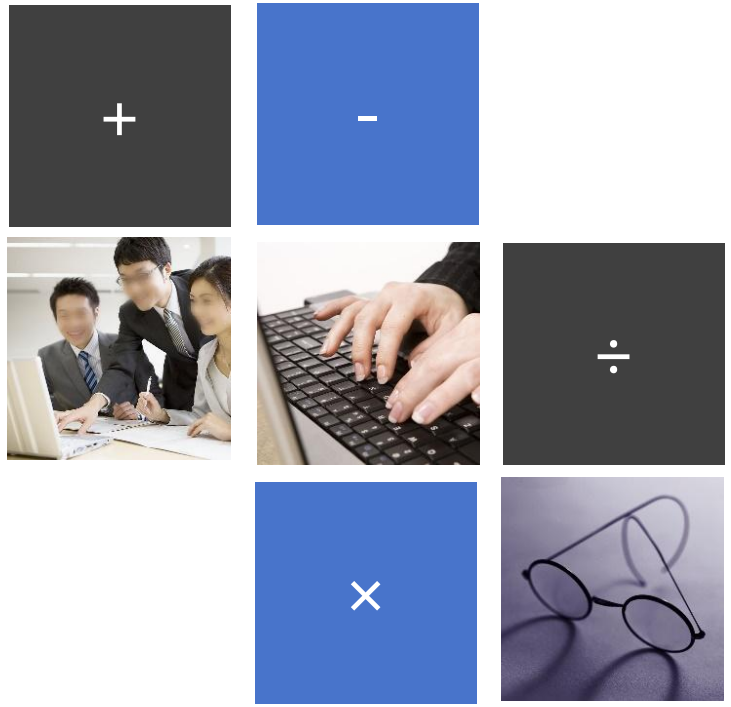
数据结构—存储结构【数据的“停车场”】

- **链式存储（如链表）：** 就像上海早高峰**找不到连续车位的巨大地下车库。**1号车停在A区，2号车停在D区，3号车停在F区。怎么办？1号车的司机手里拿了一张纸条（指针），写着“2号车在D区44座”；2号车司机拿着纸条写着3号车的位置。

算法与算法分析

■ 算法的定义

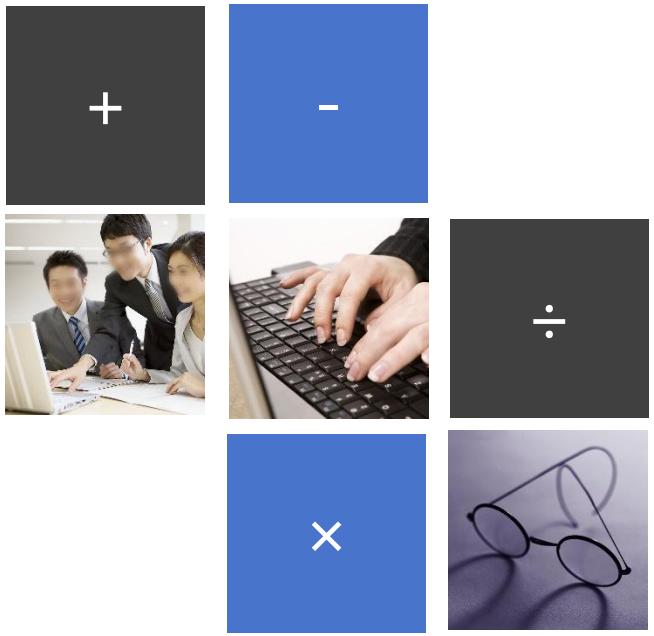
算法(Algorithm)：是对特定问题求解方法(步骤)的一种描述，是指令的有限序列，其中每一条指令表示一个或多个操作。算法具有以下五个特性:



- 有穷性： 一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。
- 确定性：算法中每一条指令必须有确切的含义。不存在二义性。且算法只有一个入口和一个出口。
- 可行性： 一个算法是能行的。即算法描述的操作都可以通过已经实现的基本运算执行有限次来实现。
- 输入： 一个算法有零个或多个输入，这些输入取自于某个特定的对象集合。
- 输出： 一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。

算法与算法分析

■ 算法设计的要求:



- 正确性(Correctness)：算法应满足具体问题的需求。
- 可读性(Readability)：算法应容易供人阅读和交流。可读性好的算法有助于对算法的理解和修改。
- 健壮性(Robustness)： 算法应具有容错处理。当输入非法或错误数据时，算法应能适当地作出反应或进行处理，而不会产生莫名其妙的输出结果。
- 效率与存储量需求： 效率指的是算法执行的时间；存储量需求指算法执行过程中所需要的最大存储空间。一般地，这两者与问题的规模有关。

算法分析

- 一个好的算法首先要具备正确性，然后是健壮性，可读性，在几个方面都满足的情况下，主要考虑**算法的效率**，通过算法的效率高低来评判不同算法的优劣程度。
- 算法效率以下两个方面来考虑：

  ① 时间效率：指的是算法所耗费的时间；

  ② 空间效率：指的是算法执行过程中所耗费的存储空间。

  ③ 时间效率和空间效率有时候是矛盾的。

算法分析

■ 算法评价标准

- 时间复杂度 T(n)
- 空间复杂度 S(n)

■ 一个特定算法的“运行工作量”的大小，只依赖于问题的规模（通常用整数量n表示），或者说，它是问题规模的函数。

算法分析

■ 算法时间效率的度量

- 算法时间效率可以用依据该算法编制的程序在计算机上执行**所消耗的时间**来度量。
- 两种度量方法：
  - 事后统计：将算法实现，测算其时间和空间开销。
    - 缺点：编写程序实现算法将花费较多的时间和精力；所得实验结果依赖于计算机的软硬件等环境因素，掩盖算法本身的优劣
  - 事前分析：对算法所消耗资源的一种估算方法。【O(f(n))】重点

算法分析

■ 事前分析方法

- 一个算法的运行时间是指一个算法在计算机上运行所耗费的时间，大致可以等于计算机执行一种简单的操作（如赋值、比较、移动等）所需的**时间**与算法中进行的简单操作**次数**乘积。
  - 算法运行时间 = 一个简单操作所需的时间x简单操作次数

- 也即算法中每条语句的执行时间之和

又称**语句频度**

算法运行时间= ∑每条语句的执行次数 x 该语句执行一次所需的时间

算法分析

算法运行时间= ∑每条语句的执行次数 x 该语句执行一次所需的时间

每条语句执行一次所需的时间，一般是随**机器**而异的。取决于机器的指令性能、速度以及编译的代码质量。是由机器本身软硬件环境决定的，它与算法无关。

所以，我们可**假设执行每条语句所需的时间均为单位时间**。此时对算法的运行时间的讨论就**可转化为讨论该算法中所有语句的执行次数**，即频度之和了。

这就可以独立于不同机器的软硬件环境来分析算法的时间性能了。

算法分析

- 例如：两个n x n 矩阵相乘的算法可描述为：

```
for(i=1;i<=n;i++)                          // 该语句判断了n+1次
    for(j=1;j<=n;j++){                     // (n+1)*n
        c[i][j]=0;                         //  n*n
        for(k=1;k<=n;k++)                  //  n*n*(n+1)
            c[i][j]=c[i][j]+a[i][k]*b[k][j];   //n*n*n
    }
```

- 我们把算法所耗费的时间定义内**该算法中每条语句的频度之和**，则上述算法的时间消耗T(n)为：

$$T(n)= 2n^3 + 3n^2 +2n+ 1$$

这是一个关于n的函数

算法分析

算法时间复杂度的渐进表示法

- 为了便于比较不同算法的时间效率，我们仅比较它们的数量级

例如：两个不同的算法，时间消耗分别是：

$T_1(n)=10n^2$ 与 $T_2(n)=5n^3$

哪个好?

- 若有某个辅助函数 f(n)，使得当n趋近于无穷大时，T(n)/f(n) 的极限值为**不等于零的常数**，则称f(n)是T(n)的同数量级函数。记作 T(n) = O(f(n))

称O(f(n))为**算法的渐进时间复杂度**（O是数量级的符号），简称**时间复杂度。**

算法分析

算法时间复杂度的渐进表示法

- 对于刚才的矩阵相乘问题， 算法耗费时间为

$$T(n)= 2n^3 + 3n^2 +2n+ 1$$

- $N \to \infty$ 时， $T(n) / n^3 =2$ ，这表示n充分大时， $T(n)$ 与 n^3 是同阶或同数量级，引入大“O”记号，则 $T(n)$ 可记作：

$$T(n) = O(n^3)$$

这就是求解矩阵相乘问题的算法的渐进时间复杂度

- 一般情况下，不必计算所有操作的执行次数，而**只考虑算法中基本操作执行的次数**，它是问题**规模 n** 的某个函数，用 $T(n)$ 表示。

算法分析

◆常见的时间复杂度

O(1)： 常量时间阶

O (n)：线性时间阶

O(logn)： 对数时间阶

O(nlogn)： 线性对数时间阶

O (n^k)： $k \geq 2$， k次方时间阶

算法分析

■ 算法分析的规则

单个操作一般视为O(1)时间；

顺序结构的语句操作时间为累加关系；

循环结构的语句时间复杂度和循环次数相关；

多层循环结构的语句时间复杂度为各层循环次数之积；

递归程序的时间复杂度为所有调用的时间复杂度的总和。

算法分析

■ **例1：**

① x += 1
s = 0

- 基本操作是加法及赋值
- $T(n)=O(2)=O(1)$

基本操作执行次数通常是
问题规模n的某个函数f(n)。

算法分析

■ **例2：**

```
② for i in range(1, n + 1):
        x += 1
        s += x
```

- 问题规模为n，基本操作是加法及赋值
- 操作次数为$f(n) = n+n$
- $T(n)=O(f(n))=O(n+n)=O(2n)=O(n)$

算法分析

■ **例3：**

```
③ for i in range(1, n + 1):
      for j in range(1, n + 1):
          x += 1
          s += x
```

- 问题规模为n，基本操作是加法及赋值
- 操作次数为 $f(n) = n*n+n*n$
- $T(n)=O(f(n))=O(2n^2)=O(n^2)$

算法分析

■ **例4：**

```
for i in range(1, n):                # i = 1,2,...,n-1
    y = y + 1                        # 语句1
    for j in range(0, 2*n + 1):      # j = 0,1,...,2n
        x += 1                       # 语句2
```

- 问题规模为n，基本操作是加法及赋值
- 操作次数为f(n)= 语句1：(n-1) + 语句2 :(n-1)(2n+1)
- T(n)=O(f(n))=O((n-1) + (n-1)(2n+1))=O($2n^2-2$)=O(n^2)

算法分析

■ **例5：**

```
i= 1            #1
while (i<=n)
        i = i * 2    #2
```

关键是要找出来执行次数 x 与 n 的关系，并表示成 n 的函数

- 问题规模为n，基本操作是乘法及赋值
- 设操作次数为x ，初始 $i = 1$，
- 第一次循环后 $i = i * 2 = 2^1$
- 第二次循环后 $i = i * 2 = 2 * 2 = 2^2$
- 第三次循环后 $i = i * 2 = 2^2 * 2 = 2^3$
- 第 x 次循环后 $i = 2^x$
- 由循环条件 $i <= n$，可得 $2^x <= n$
- 可得 $x <= \log_2 n$
- 取最大值，$f(n) = \log_2 n$
- $T(n) = O(f(n)) = O(\log_2 n) = O(\log n)$

2.5 算法与算法分析

$O(1)<O(\log n)<O(n)<O(n\log n)<O(n^2)<O(n^3)$

- 指数时间的关系为：

$O(2^n)<O(n!)<O(n^n)$

空间复杂度(Space complexity)：是指算法编写成程序后，在计算机中运行时所需存储空间大小的度量。记作： $S(n)=O(f(n))$，其中： n为问题的规模(或大小).

该存储空间一般包括三个方面：

1. 指令常数变量所占用的存储空间;
2. 输入数据所占用的存储空间;
3. 辅助(存储)空间。

一般地，算法的空间复杂度指的是辅助空间。

1. 一维数组a[n]： 空间复杂度 O(n)
2. 二维数组a[n][m]： 空间复杂度 O(n*m)

怎么才能学好这门课程呢?

- 勤于思考
- 多做练习
- 善于寻求帮助
- 不怕苦难，不放弃！！！

Thank you !